



Universidad de
SanAndrés

**I302 - Aprendizaje Automático
y Aprendizaje Profundo**

**Trabajo Práctico 4:
Aprendizaje No-Supervisado**

Manuel Borrell

30 de mayo de 2025

Ingeniería en Inteligencia Artificial

1. Clustering de datos

1.1. Introducción

En esta primera parte del trabajo, se analiza un conjunto de datos sin etiquetas, compuesto por 4999 muestras y dos características desconocidas. El objetivo es aplicar diferentes algoritmos de agrupamiento para descubrir posibles estructuras naturales presentes en los datos. Esta tarea es fundamental en contextos donde no se dispone de una variable objetivo, como segmentación de clientes, detección de anomalías o exploración de datos no etiquetados.

La entrada de cada algoritmo es un conjunto de puntos $\mathbf{x}_i \in R^2$ y la salida es una asignación de etiquetas de clúster para cada punto. Se implementaron y compararon los métodos K-Means, Gaussian Mixture Models (GMM) y DBSCAN, evaluando su comportamiento ante diferentes formas y distribuciones de los datos.

1.2. Métodos

K-Means

Este algoritmo busca minimizar la suma de distancias entre cada punto y el centroide de su clúster:

$$L = \sum_{i=1}^N \min_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \mu_k\| \quad (1)$$

K-Means requiere que se indique la cantidad de clusters K previamente.

Gaussian Mixture Model (GMM)

GMM modela los datos como una combinación de distribuciones gaussianas, permitiendo mayor flexibilidad que K-Means al capturar formas elípticas en los clusters. Se utiliza el algoritmo EM (Expectation-Maximization) para estimar los parámetros del modelo:

- **E-step:** calcular la responsabilidad de cada componente para cada punto.
- **M-step:** actualizar medias, covarianzas y pesos.

La inicialización se realizó con los centroides de K-Means y matrices de covarianza diagonales.

DBSCAN

DBSCAN identifica regiones densas en el espacio de datos mediante dos parámetros: el radio ε y la cantidad mínima de puntos *minPts* para considerar un vecindario denso. A diferencia de K-Means y GMM, no requiere indicar la cantidad de clusters y permite detectar ruido de forma natural.

Preprocesamiento

Todos los datos fueron normalizados usando estandarización Z-score para garantizar que ambas variables tuvieran media cero y varianza unitaria, evitando que alguna domine la métrica de distancia.

Selección de hiperparámetros

K-Means: se eligió $K = 15$ con el método del codo tras evaluar $K \in [1, 100]$. Utilizando como metrica las distancias al centroide. (fig 1)

GMM: se utilizó $K = 26$ utilizando el mismo método para encontrar el hiperparámetro.(fig 2)

DBSCAN: se exploraron combinaciones de $\varepsilon \in [0,05, 0,5]$ y $minPts \in [3, 10]$. Se aplicó un grid search y llegue al resultado de que $minPts = 10$ y $\varepsilon = 0,1$.

1.3. Resultados

K-Means

K-Means generó clusters compactos y bien separados, aunque con formas algo rígidas. algun par de los culsters podrian ser resumidos en uno solo. (fig 3)

GMM

GMM ofreció segmentaciones más suaves y naturales, permitiendo solapamiento entre clusters y logra explicar los culsters que kmeans no pudo. (fig 4)

1.4. DBSCAN

DBSCAN detectó clusters con formas irregulares y aisló ruido, pero fue sensible a los hiperparámetros. los resultados son parecidos a los que podriamos llegar si analizamos el dataset a ojo. (fig 5)

Conclusión

Ya que no todos los modelos los podemos medir bajo la misma metrica, entre k means y gmm podemos ver que gmm al converger en mas clusters va a dar una mejor suma de distancias que k means

Método	K	Pérdida (Loss)
K-Means	15	1060
GMM	26	935
GMM	15	1287

Como vemos en la tabla, a misma cantidad de clusters k means resulta en una mejor loss y si sumamos 11 clusters la diferencia en loss no es suficiente por lo que k means dio mejores resultados que gmm.

2. Reducción dimensional

2.1. Introducción

Esta parte explora cómo representar los datos en espacios de menor dimensión para facilitar su visualización. Se aplicaron PCA (Análisis de Componentes Principales) y variational autoencoders (redes neuronales para compresión).

La entrada del algoritmo es el dataset MNIST el cual consta de 70000 imágenes de 28x28 pixeles, las imágenes son de números escritos a mano, la salida del sistema es una transformación la cual reduce la cantidad de dimensiones necesarias para representar cada imagen.

2.2. Métodos

PCA

PCA proyecta los datos en las direcciones de mayor varianza. Se retuvieron la cantidad de componentes principales necesarios para mantener un 80 % de la variabilidad.

$$Z = XW, \quad W = \text{autovectores de la matriz de covarianza de } X \quad (2)$$

VAE

Se entrenó una red neuronal con codificador y decodificador simétrico para mapear los datos a una dimension latente menor. La función de pérdida utilizada fue el error cuadrático medio (MSE) entre entrada y reconstrucción, con un término agregado correspondiente a la divergencia Kullback-Leibler (KL), la cual actúa como regularizador. Este término fuerza a la distribución latente aprendida a aproximarse a una distribución normal estándar, promoviendo una representación latente continua y estructurada.

Preprocesamiento

Todos los datos fueron normalizados dividiendo por 255 ya que cada variable es una escala de grises de 0 a 255

Selección de hiperparámetros

PCA: se eligió 43 componentes principales ya que estos mantenían la variabilidad deseada. En esta figura podemos ver el mse a medida que suben los componentes (fig 6)

VAE: la dimension latente seleccionada fue de 43 para poder hacer la comparación con PCA

2.3. Resultados

PCA

PCA logro reducir la dimensionalidad a 43 componentes y los números todavía pueden entenderse perfectamente, la imagen queda levemente difuminada(fig 7).

VAE

VAE reconstruye a un nivel mucho mejor que pca las imágenes son perfectamente claras y muestra pocos indicios de haber sido transformadas previamente (fig 7)

Conclusión

Ambas transformaciones lograron reducir la dimensionalidad manteniendo una imagen entendible y clara pero el vae es claramente superior esto tambien lo podemos ver al observar el espacio latente proyectado a sus dos mejores coordenadas(fig 8,9). En el grafico de pca (fig 8) podemos ver que hay mucho solapamiento y no hay una clara división entre clusters debido a la linealidad de pca, muchos reconocen esta linealidad como uno de sus mejores puntos y una de sus mayores debilidades al mismo tiempo, gracias a esta linealidad el modelo se puede usar sin entrenar con unas simples multiplicaciones pero cuando los dataset son muy complejos las transformaciones lineales no son suficientes para separar estos clusters de manera correcta.

En cambio, el VAE (Fig. 9) sí logra una organización mucho más clara del espacio latente. Como puede aprender relaciones no lineales y además genera una representación probabilística del espacio de características, es capaz de capturar mejor la estructura interna de los datos. Esto hace que las clases queden mucho más separadas y que las distribuciones latentes tengan más sentido.

2.4. Extra

Como extra, probé usar el *decoder* del VAE para generar muestras nuevas. Para eso, tomé vectores aleatorios del espacio latente muestreados desde una distribución normal estándar, es decir, $\mathcal{N}(0, I)$. Estos vectores representan puntos en el espacio latente que el VAE aprendió durante el entrenamiento.(fig 10)

3. figuras

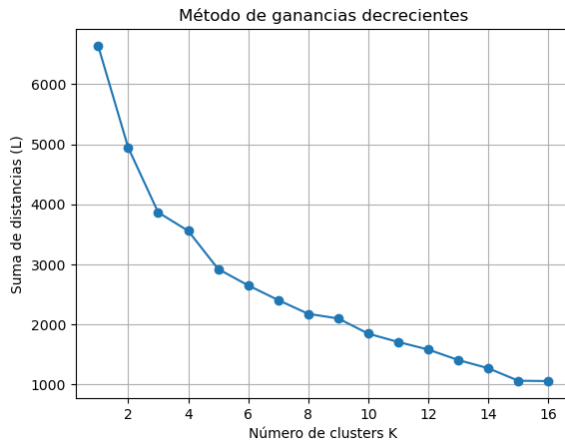


Figura 1: Método de ganancias decrecientes, Kmeans

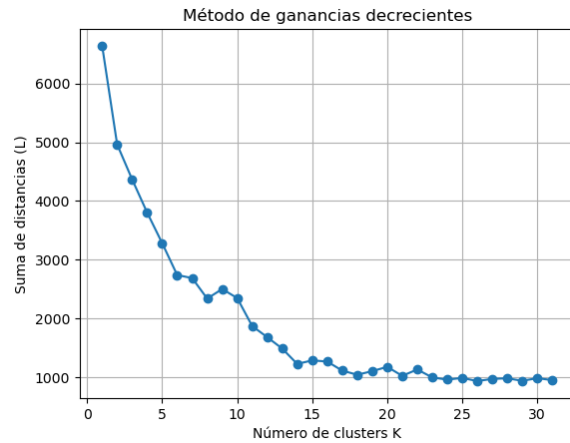


Figura 2: Método de ganancias decrecientes, GMM

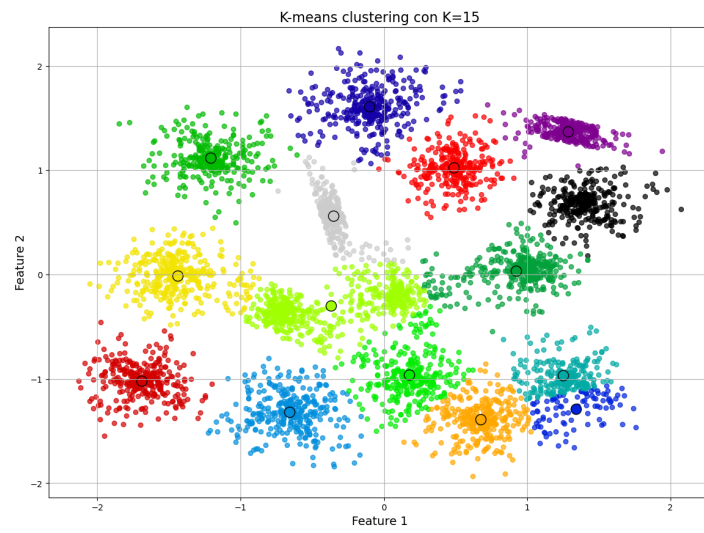


Figura 3: Resultados, Kmeans

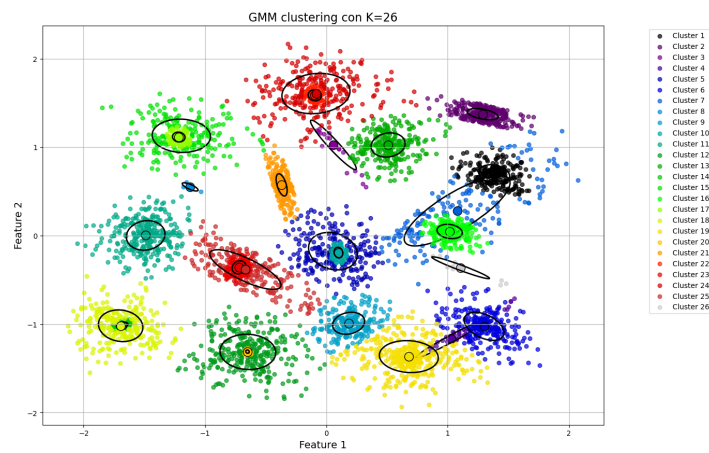


Figura 4: Resultados, GMM

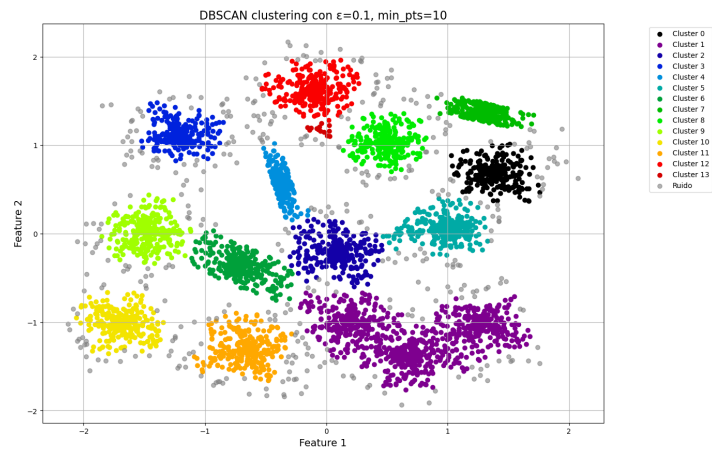


Figura 5: Resultados, DBSCAN

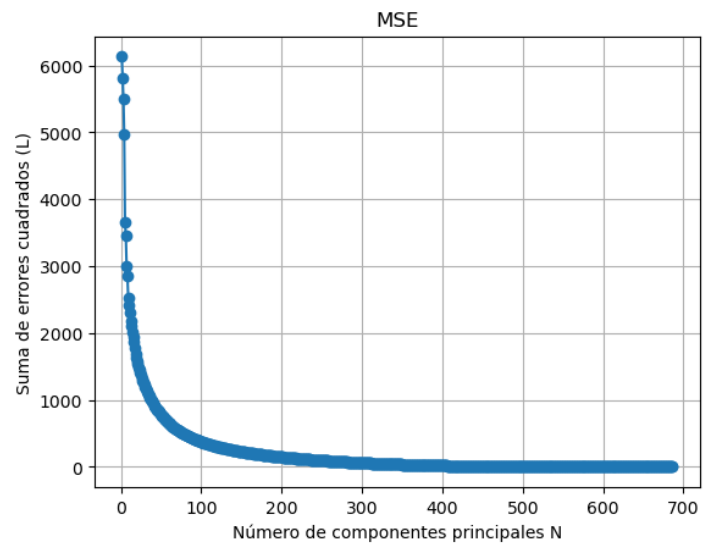


Figura 6: MSE vs componentes principales

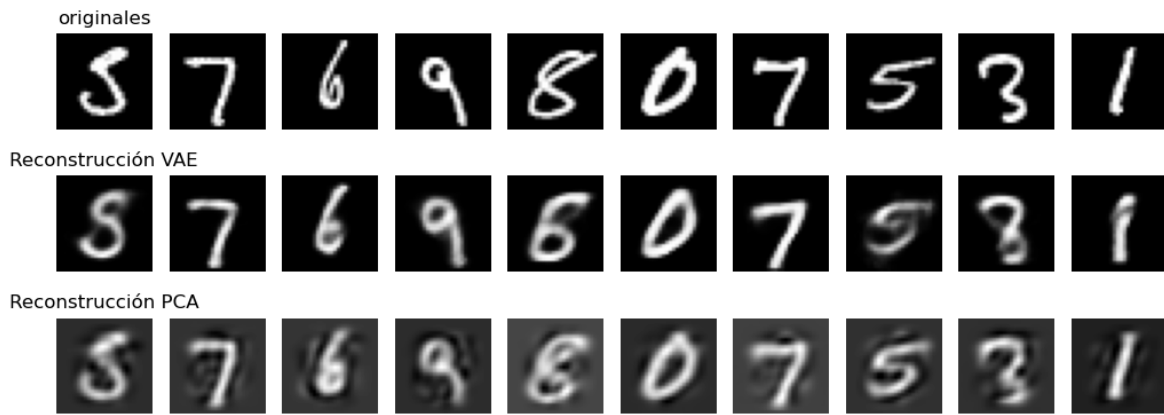


Figura 7: Resultados, VAE y PCA

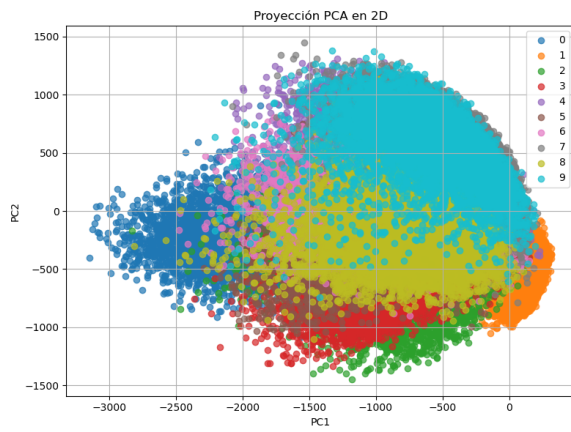


Figura 8: Proyección con 2 componentes principales

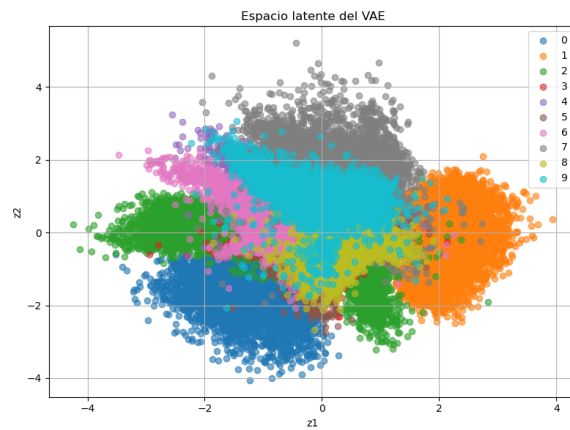


Figura 9: Espacio latente con $n = 2$

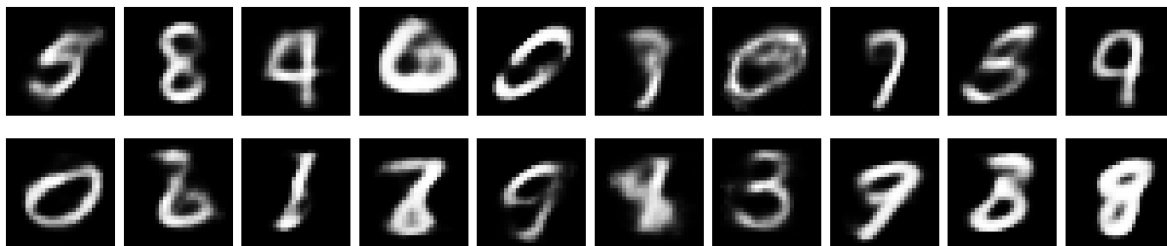


Figura 10: Muestras nuevas con decoder